

TEAVS-ADCRS Two-Way Communication Verification

1. Purpose

The purpose of this work was to establish and verify a traceable two-way communication path between TEAVS and the ADCRS/core-model integration within PHOENIX.

The main issue identified was that TEAVS could submit data to the core integration endpoint, but the original response only confirmed that the request had been accepted.

The original response was:

```
{  
  "status": 202,  
  "message": "Core model integration data sent successfully"  
}
```

This did not provide TEAVS with a way to identify the specific integration request later or retrieve the corresponding ADCRS result.

To solve this, an `integration_event_id` was introduced as a correlation identifier across the integration flow.

2. Communication Design

The updated communication flow is:

TEAVS → API Gateway → RabbitMQ → data-ingestion-service →
`coreModelIntegration()` → IntegrationLog

The return path is:

TEAVS → GET integration using integration_event_id → PHOENIX backend → stored ADCRS/core-model result

The same integration_event_id is used throughout the request lifecycle.

This provides a direct link between the original TEAVS submission and the downstream processing result.

3. Submission Endpoint

TEAVS submits an event through:

POST /api/ingestion/core

The request must contain a valid JWT and must come from an authorised role.

The current authorised roles are:

- admin
- ingestion_service

The request is also subject to:

- JWT authentication
- RBAC
- rate limiting
- payload validation

4. Integration Event Identifier

The integration payload interface was updated to include:

```
integration_event_id: string;
```

The relevant interface is:

```
export interface CoreModelIntegrationPayload {  
  integration_event_id: string;  
  url: string;  
  text: string;  
  timestamp: string;  
  hazard_type: string;  
  hazard_severity: number;  
  hazard_timestamp: string;  
  hazard_location: string;  
  hazard_status: string;  
  alert_level: string;  
  source: string;  
}
```

The integration_event_id acts as the correlation identifier for the complete TEAVS-ADCRS transaction.

5. ID Generation at the API Gateway

The API Gateway now generates the integration identifier before the message is sent to RabbitMQ.

The identifier is created using a UUID:

```
const integrationEventId = crypto.randomUUID();
```

The generated ID is then included in the payload:

```
const body = {  
  ...req.body,  
  integration_event_id: integrationEventId,  
} as CoreModelIntegrationPayload;
```

This ensures that the identifier exists before the message enters the asynchronous RabbitMQ workflow.

6. Updated POST Response

The API Gateway now returns the generated identifier to TEAVS.

Example response:

```
{  
  "status": 202,  
  "message": "Core model integration data sent successfully",  
  "integration_event_id": "7956d9bd-83be-45dd-8472-4583b189043a"  
}
```

This means TEAVS now receives a unique reference for each accepted integration request.

7. RabbitMQ Communication

The same payload, including the generated `integration_event_id`, is published to:

`core_model_integration_queue`

The data-ingestion-service consumes the message and forwards the payload to:

coreModelIntegration()

This ensures that the same identifier generated at the API Gateway is preserved throughout the asynchronous communication path.

8. Integration Log Correlation

The downstream coreModelIntegration() service was updated so that the existing IntegrationLog record uses the identifier supplied in the payload.

The integration log is created using:

```
const integrationLog = await IntegrationLog.create({
  integration_event_id: payload.integration_event_id,
  integration_type: IntegrationType.CORE,
  input: JSON.stringify(payload),
  status: IntegrationStatus.CREATED,
});
```

Previously, the database model generated a new UUID automatically.

Using the identifier supplied by the API Gateway ensures that TEAVS and the backend reference the same integration record.

9. Result Retrieval Endpoint

PHOENIX already contained an integration retrieval endpoint:

GET /api/users/integration/:integrationId

The route is protected by JWT authentication.

The backend retrieves the record using:

```
IntegrationLog.findByPk(dto.integration_event_id)
```

The returned integration object contains:

- integration_event_id
- integration type
- original input
- output
- processing status
- note
- creation timestamp
- update timestamp

This provides the return path required for TEAVS to retrieve ADCRS processing results.

10. Two-Way Communication Flow

The verified communication pattern is now:

Step 1 – TEAVS submits an event

TEAVS sends:

```
POST /api/ingestion/core
```

Step 2 – PHOENIX generates a correlation ID

The API Gateway generates:

```
integration_event_id
```

Step 3 – TEAVS receives the ID

Example:

7956d9bd-83be-45dd-8472-4583b189043a

Step 4 – Request enters RabbitMQ

The same identifier is included in the message sent to:

core_model_integration_queue

Step 5 – ADCRS/core processing uses the same ID

The downstream service creates the integration record using the supplied integration_event_id.

Step 6 – Result is stored against the same record

When inference completes, the result is written to:

IntegrationLog.output

Step 7 – TEAVS retrieves the result

TEAVS calls:

GET /api/users/integration/{integration_event_id}

The backend returns the matching integration record.

11. Runtime Verification

A valid TEAVS integration request was submitted through:

POST /api/ingestion/core

The API returned:

HTTP 202 Accepted

and provided the identifier:

7956d9bd-83be-45dd-8472-4583b189043a

The same identifier was then used to request:

GET /api/users/integration/7956d9bd-83be-45dd-8472-4583b189043a

The backend returned:

HTTP 200 OK

and returned an integration record containing the same identifier.

This confirmed that the correlation ID is successfully preserved between the submission and retrieval stages.

12. Verification Result

Verification Item	Result
TEAVS event submission accepted	PASS
Integration ID generated	PASS
Integration ID returned to TEAVS	PASS
Same ID included in RabbitMQ payload	PASS
Same ID stored in IntegrationLog	PASS

Integration record retrievable using ID	PASS
Retrieved record contains original input	PASS
Current processing status retrievable	PASS
Final model output available	BLOCKED by missing model asset

13. Current Retrieved Result

The integration record was successfully retrieved using the generated identifier.

The returned record showed:

status: error

note: Training model not found

output: {}

This does not indicate a communication failure.

The result confirms that the request was successfully correlated and retrieved, but downstream inference could not complete because the configured trained model artifact was not available in the local environment.

14. Remaining Model Dependency

The backend expects the model:

final_core_xgb_xgboost_trey_xgb_core_v2_epoch_100.joblib

The filename is referenced in both:

- `libs/common/src/helper/core_model_integration.py`
- `data-ingestion-service/src/services/ingestion.service.ts`

No `.joblib`, `.pkl`, or `.pickle` model artifact was found in the local repository.

The communication path therefore works independently of the missing inference dependency.

15. Intended ADCRS Response

Once the trained model is available and inference completes, the integration output can be stored in:

`IntegrationLog.output`

The result may contain fields such as:

- risk score
- risk level
- detected threat or anomaly
- confidence
- recommended response

TEAVS can then retrieve that output using the previously returned `integration_event_id`.

16. Evidence

EVID-COMM-01 – Integration ID returned

```
● (base) abdurrehman@Abduls-Air backend % export INGESTION_TOKEN=$(curl -s -X POST http://localhost:3001/api/users/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "username": "rbac_test_ingestion",
    "password": "PhoenixTemp2026!Y"
  }' | python3 -c 'import sys,json; print(json.load(sys.stdin)["access_token"])')
● (base) abdurrehman@Abduls-Air backend % curl -i -X POST http://localhost:3001/api/ingestion/core \
  -H "Authorization: Bearer $INGESTION_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "url": "https://example.com/comm-evidence",
    "text": "TEAVS COMMUNICATION EVIDENCE",
    "timestamp": "2026-09-27T10:15:00+10:00",
    "hazard_type": "phishing",
    "hazard_severity": 4,
    "hazard_timestamp": "2026-09-27T10:14:00+10:00",
    "hazard_location": "VIC",
    "hazard_status": "active",
    "alert_level": "critical",
    "source": "TEAVS"
  }',
HTTP/1.1 202 Accepted
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 134
ETag: W/"86-pjCB0Ghtx0M0k6DXRD5EjtEekCRA"
Date: Sun, 27 Sep 2026 00:13:19 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":202,"message":"Core model integration data sent successfully","integration_event_id":"3c43150e-5bc7-4755-9f41-e2d1f73484b5"}
● (base) abdurrehman@Abduls-Air backend %
```

EVID-COMM-02 – Integration record retrieved

```
(base) abdurrehman@Abduls-Air backend % curl -i \
  -H "Authorization: Bearer $INGESTION_TOKEN" \
  http://localhost:3001/api/users/integration/3c43150e-5bc7-4755-9f41-e2d1f73484b5
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 758
ETag: W/"2f6-RXJfCm00b19Cv0Bfrr9ZB/mwQ"
Date: Sun, 27 Sep 2026 00:15:42 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":200,"message":"Integration fetched successfully","integration":{"integration_event_id":"3c43150e-5bc7-4755-9f41-e2d1f73484b5","integration_type":"core","input":{"url":"https://example.com/comm-evidence","text":"TEAVS COMMUNICATION EVIDENCE","timestamp":"2026-09-27T10:15:00+10:00","hazard_type":"phishing","hazard_severity":4,"hazard_timestamp":"2026-09-27T10:14:00+10:00","hazard_location":"VIC","hazard_status":"active","alert_level":"critical","source":"TEAVS"},"integration_event_id":"3c43150e-5bc7-4755-9f41-e2d1f73484b5"},"output":{},"status":"error","note":"Training model not found","created_at":"Sun Sep 27 2026 00:13:19 GMT+0000 (Coordinated Universal Time)","updated_at":"Sun Sep 27 2026 00:13:19 GMT+0000 (Coordinated Universal Time)"}
(base) abdurrehman@Abduls-Air backend %
```

EVID-COMM-03 – Current processing status

```
(base) abdurrehman@Abduls-Air backend % curl -i \
  -H "Authorization: Bearer $INGESTION_TOKEN" \
  http://localhost:3001/api/users/integration/3c43150e-5bc7-4755-9f41-e2d1f73484b5
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 758
ETag: W/"2f6-RXJfCm00b19Cv0Bfrr9ZB/mwQ"
Date: Sun, 27 Sep 2026 00:15:42 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":200,"message":"Integration fetched successfully","integration":{"integration_event_id":"3c43150e-5bc7-4755-9f41-e2d1f73484b5","integration_type":"core","input":{"url":"https://example.com/comm-evidence","text":"TEAVS COMMUNICATION EVIDENCE","timestamp":"2026-09-27T10:15:00+10:00","hazard_type":"phishing","hazard_severity":4,"hazard_timestamp":"2026-09-27T10:14:00+10:00","hazard_location":"VIC","hazard_status":"active","alert_level":"critical","source":"TEAVS"},"integration_event_id":"3c43150e-5bc7-4755-9f41-e2d1f73484b5"},"output":{},"status":"error","note":"Training model not found","created_at":"Sun Sep 27 2026 00:13:19 GMT+0000 (Coordinated Universal Time)","updated_at":"Sun Sep 27 2026 00:13:19 GMT+0000 (Coordinated Universal Time)"}
(base) abdurrehman@Abduls-Air backend %
```

17. Security and Engineering Value

The use of a shared integration identifier improves the design in several ways.

It provides:

- traceability between TEAVS requests and ADCRS processing
- reliable correlation across asynchronous RabbitMQ communication
- a way for TEAVS to retrieve processing results
- improved debugging and incident investigation
- clearer audit records
- support for future idempotency controls
- a foundation for monitoring integration status

The identifier also separates request acceptance from model-processing completion, which is appropriate for an asynchronous integration architecture.

18. Final Communication Architecture

The resulting architecture is:

TEAVS

↓

POST /api/ingestion/core

↓

Generate integration_event_id

↓

JWT + RBAC + Rate Limit + Validation

↓

RabbitMQ

↓

data-ingestion-service

↓

coreModelIntegration()

↓

IntegrationLog

↓

ADCRS/core-model output

↓

GET /api/users/integration/{integration_event_id}

↓

TEAVS retrieves result

19. Conclusion

The TEAVS–ADCRS communication path now supports request correlation in both directions.

TEAVS can submit a valid event and receive a unique integration_event_id.

The same identifier travels through RabbitMQ and is stored with the downstream integration record.

TEAVS can later use that identifier to retrieve the corresponding processing status and ADCRS/core-model output.

Runtime testing confirmed that the same identifier generated during submission could be successfully used to retrieve the correct integration record through the PHOENIX backend.

The remaining limitation is the unavailable trained model artifact, which prevents final inference output from being produced in the current local environment. The communication and correlation mechanism itself has been successfully implemented and verified.